High-Level Design (HLD)

RAG-Based Customer Support Assistant with LangGraph & HITL

RAG Customer Support Assistant | v1.0 | Innomatics Research Labs

1. System Overview

This system is a Retrieval-Augmented Generation (RAG) based Customer Support Assistant. It processes a PDF knowledge base, retrieves relevant information using vector embeddings, and answers user queries contextually. When the system lacks sufficient context to answer confidently, it escalates the query to a human agent via the Human-in-the-Loop (HITL) mechanism.

Problem Definition

- Traditional customer support bots rely on static rule-based logic and fail on nuanced queries.
- LLMs hallucinate when answering without grounded context.
- There is no fallback mechanism when AI confidence is low.

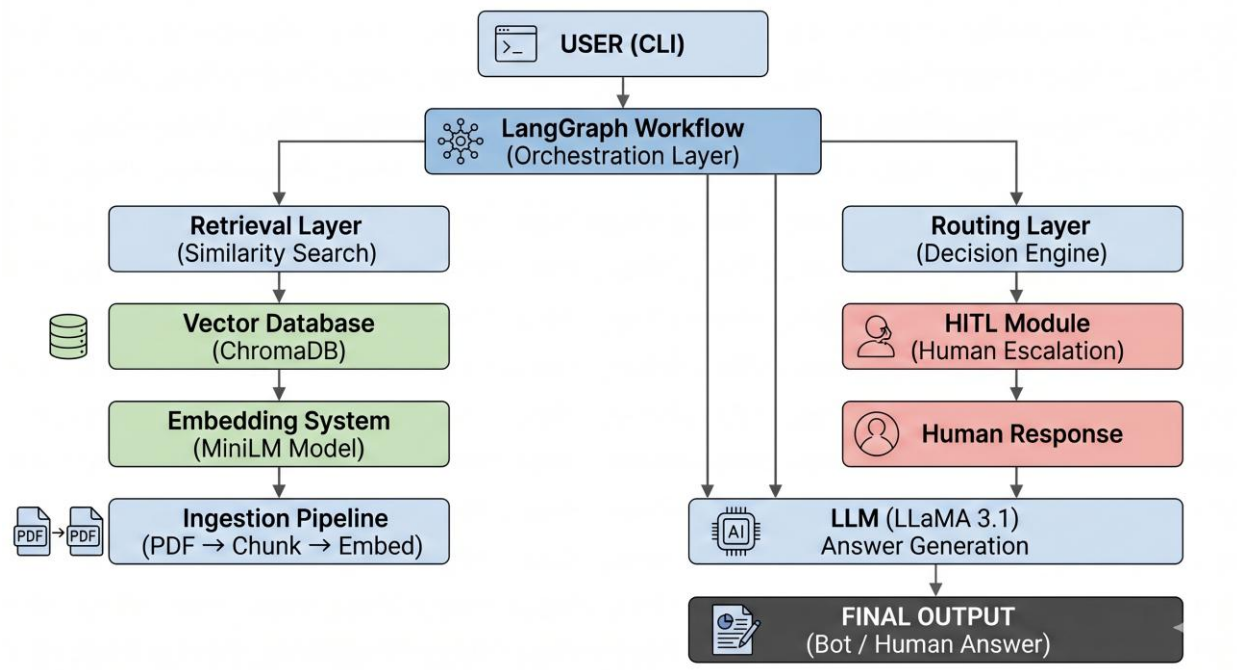
Scope

- Single PDF knowledge base ingestion and retrieval.
- Natural language question answering using RAG pipeline.
- Graph-based workflow orchestration using LangGraph.
- Conditional routing based on answer confidence.
- Human-in-the-Loop escalation with audit logging.

2. Architecture Overview

Layer	Component	Technology
User Interface	CLI chat loop	Python (main.py)
Ingestion Pipeline	PDF Loader + Chunker	PyPDF + LangChain RecursiveTextSplitter
Embedding System	Text Embeddings	HuggingFace all-MiniLM-L6-v2 (local)
Vector Database	ChromaDB	Chroma (persisted on disk)
Retrieval Layer	Similarity Search	LangChain Retriever (top-k=3)
LLM Layer	Answer Generation	Groq LLaMA-3.1-8B (free API)
Workflow Engine	Graph Orchestration	LangGraph StateGraph

Routing Layer	Conditional Routing	Phrase-based confidence check
HITL Module	Human Escalation	Terminal input + escalation_log.txt



3. Data Flow

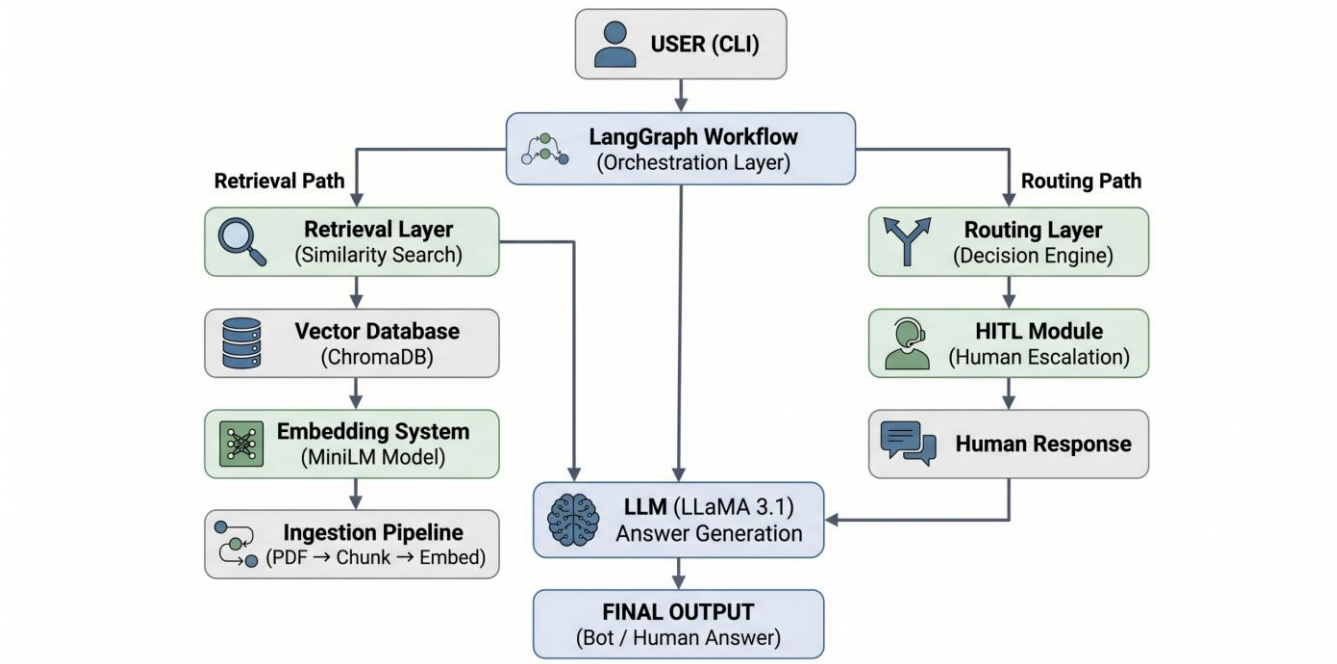
Ingestion Flow (run once)

Step	Action
1	Load PDF using PyPDFLoader
2	Split into 1000-char chunks with 200-char overlap
3	Generate embeddings using HuggingFace all-MiniLM-L6-v2
4	Store embeddings in ChromaDB (persisted to chroma_db/ folder)

Query Flow (every user question)

Step	Action
1	User types a question in the CLI
2	Question is passed into LangGraph as the initial state
3	process_node retrieves top-3 chunks from ChromaDB
4	Chunks + question are sent to Groq LLaMA LLM
5	Router checks answer for low-confidence phrases
6a	If confident: answer_node returns response to user
6b	If not confident: escalate_node triggers HITL module
7	Human agent types response; logged to escalation_log.txt

SYSTEM HLD: ORCHESTRATED RETRIEVAL & HITL ANSWER GENERATION



4. Technology Choices

Technology	Reason for Choice
ChromaDB	Lightweight, file-based vector store. No server needed.
LangGraph	Provides graph-based workflow control.
HuggingFace all-MiniLM-L6-v2	Free, fast, local embedding model (~90MB). No API key needed
Groq LLaMA-3.1-8B	Free tier API. Very fast inference. Llama 3.1 produces high-quality answers
LangChain	Provides standardized abstractions for loaders, splitters, retrievers, and LLM

5. Scalability Considerations

Concern	Current Approach	Scalable Alternative
Large documents	Single PDF, chunked at 1000 cha	sMulti-PDF ingestion pipeline, batch embedding
Query load	Single user CLI	FastAPI REST endpoint, async processing
Latency	Sequential retrieval + LLM call	Async LangGraph, response caching
Vector store	ChromaDB (local)	Pinecone / Weaviate (cloud, distributed)
LLM cost	Groq free tier	Self-hosted Ollama or paid tier with rate limiting